# Gu??a Persona 4

# Guía paso a paso – Persona 4 (Infraestructura)

**Reto 8 – Observabilidad y Monitoreo · Tú inicias la construcción**

## 0. Qué se espera de ti

Eres quien levanta la base: el stack de observabilidad, el contrato común y el API Gateway instrumentado. Las otras tres personas dependen de que termines bien esta parte, y por eso entras primero. Vale **1.0 punto** (Infraestructura) más la base de la documentación.

**Entregables tuyos**

| # | Entregable | Dónde |
|---|---|---|
| 1 | Diagrama previo de arquitectura | `docs/img/arquitectura-observabilidad.png` y README |
| 2 | Servicios de observabilidad en el compose | `docker-compose.yml` |
| 3 | Configuración de Prometheus (7 targets) y reglas | `observability/prometheus/` |
| 4 | Aprovisionamiento de Grafana | `observability/grafana/provisioning/` |
| 5 | Configuración de Loki y Promtail | `observability/loki/`, `observability/promtail/` |
| 6 | API Gateway instrumentado | `api-gateway/` |
| 7 | Contrato para el equipo | `docs/contrato-observabilidad.md` |
| 8 | Investigación y justificación en el README | `README.md` (tu sección) |

## 1. Antes de empezar

1. `git pull` y trabaja sobre `main` (decisión del equipo).
2. Comprueba cuántos recursos tienes: el stack nuevo suma unos **1–1,5 GB** de RAM además de los 7 servicios. Con 8 GB, cierra lo que no uses.
3. **Conflictos de puertos que ya existen en el proyecto** (importantes):

| Puerto | Quién lo usa hoy | Qué hacer |
|---|---|---|
| **9090** | Jenkins (`docker-compose.ci.yml`) | Publica Prometheus en el host como **`9091:9090`**. Dentro de Docker sigue siendo 9090. Documéntalo en el README, porque el reto usa 9090. |
| 3000 | Libre | Grafana `3000:3000`. |
| 9411 | Libre | Zipkin `9411:9411`. |

| Puerto | Quién lo usa hoy | Qué hacer |
|---|---|---|
| 3100 | Libre | Loki: no hace falta publicarlo. |
| 8080 | API Gateway | No tocar. |

4. Comprueba la red: ya existe y se llama `microservices-network`. **Usa esa**, no crees otra.

## 2. Orden de trabajo (checklist)

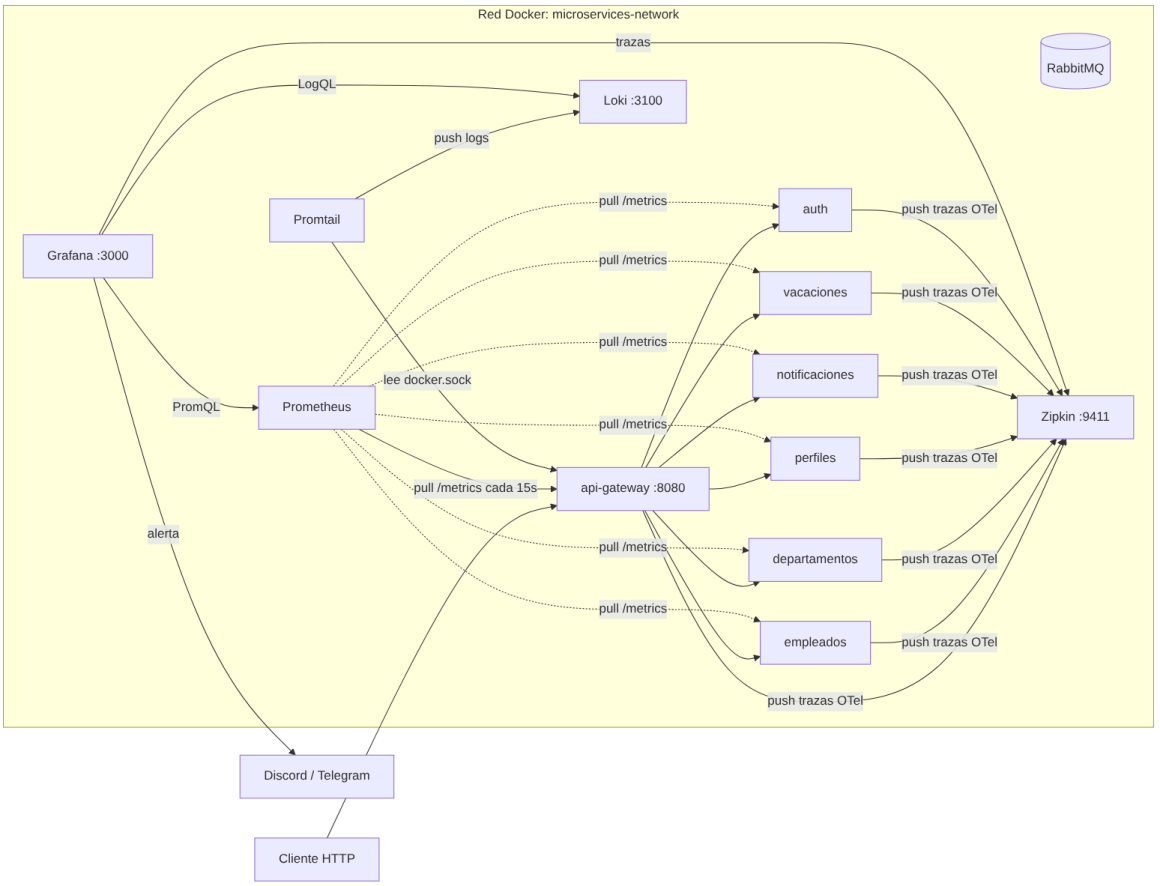
- [ ] Paso 1: Investigación y diagrama
- [ ] Paso 2: Estructura de carpetas
- [ ] Paso 3: Servicios en `docker-compose.yml`
- [ ] Paso 4: `prometheus.yml` y `rules.yml`
- [ ] Paso 5: Loki y Promtail
- [ ] Paso 6: Provisioning de Grafana
- [ ] Paso 7: Instrumentar el API Gateway
- [ ] Paso 8: Contrato común
- [ ] Paso 9: Validación y compatibilidad con la CI (Reto 7)
- [ ] Paso 10: README y entrega

## 3. Paso 1 – Investigación y diagrama (antes de implementar)

Escribe en el README, con tus palabras, estos cuatro puntos:

1. **Tres pilares:** métricas (números agregados en el tiempo: *cómo se comporta*), logs (eventos con detalle: *qué pasó*) y trazas (recorrido de una petición entre servicios: *por dónde pasó y cuánto tardó*).
2. **Pull frente a Push:** Prometheus *va a buscar* (hace scraping de `/metrics` cada 15 s). Los servicios *envían* sus trazas a Zipkin (push) y Promtail *empuja* los logs a Loki.
3. **OpenTelemetry:** estándar abierto de la CNCF para instrumentar. Es agnóstico del lenguaje y del proveedor, así que sirve igual para Java, Node, Go, Python y .NET.
4. **W3C Trace Context:** el `traceId` viaja en la cabecera HTTP `traceparent`. Cualquier lenguaje que la respete continúa la misma traza.

**Diagrama.** Este es el borrador (lo puedes rehacer en draw.io si prefieres). Cumple lo que pide el reto: componentes, red de Docker, flujo de trazas y consumo de Grafana.




Arquitectura de observabilidad

**Zipkin o Jaeger – elige Zipkin y justifícalo:** es un solo contenedor liviano, coincide con los exportadores del reto (`exporter-zipkin` en cada lenguaje), se levanta en memoria sin base de datos y Grafana lo lee como datasource. Jaeger sería más completo, pero consume más y no aporta nada para este alcance.

## 4. Paso 2 – Estructura de carpetas

```
observability/
├── prometheus/
│   ├── prometheus.yml
│   └── rules.yml
├── loki/
│   └── loki-config.yml
├── promtail/
│   └── promtail-config.yml
└── grafana/
    └── provisioning/
        ├── datasources/datasources.yml
        └── dashboards/
            ├── dashboards.yml
            └── (aquí P1 pondrá el JSON del dashboard)
docs/contrato-observabilidad.md
```

## 5. Paso 3 – Servicios en `docker-compose.yml`

Añade esto **dentro de** `services:` y el volumen nuevo en `volumes:`. Cambia las versiones si alguna etiqueta no existe: **no uses** `latest`, para que todos los equipos ejecuten lo mismo.

```yaml
  prometheus:
    image: prom/prometheus:v2.55.1
    container_name: prometheus
    ports:
      - "9091:9090"            # 9090 está ocupado por Jenkins en el host
    volumes:
      - ./observability/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./observability/prometheus/rules.yml:/etc/prometheus/rules.yml:ro
    networks: [microservices-network]
    restart: unless-stopped

  zipkin:
    image: openzipkin/zipkin:3.4.2
    container_name: zipkin
    ports:
      - "9411:9411"
    environment:
      JAVA_OPTS: "-Xmx256m"    # Zipkin es JVM; acota la memoria
    networks: [microservices-network]
    restart: unless-stopped

  loki:
    image: grafana/loki:3.2.1
    container_name: loki
    command: -config.file=/etc/loki/loki-config.yml
    volumes:
      - ./observability/loki/loki-config.yml:/etc/loki/loki-config.yml:ro
      - loki-data:/loki
    networks: [microservices-network]
    restart: unless-stopped

  promtail:
    image: grafana/promtail:3.2.1
    container_name: promtail
    command: -config.file=/etc/promtail/promtail-config.yml
    volumes:
      - ./observability/promtail/promtail-config.yml:/etc/promtail/promtail-
config.yml:ro
      - /var/run/docker.sock:/var/run/docker.sock:ro
    depends_on: [loki]
    networks: [microservices-network]
    restart: unless-stopped

  grafana:
    image: grafana/grafana:11.3.0
    container_name: grafana
    ports:
      - "3000:3000"
    environment:
      GF_SECURITY_ADMIN_USER: admin
```

```
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_ADMIN_PASSWORD:-admin}
    volumes:
      - grafana-data:/var/lib/grafana                        # persiste dashboards
      - ./observability/grafana/provisioning:/etc/grafana/provisioning:ro
    depends_on: [prometheus, loki, zipkin]
    networks: [microservices-network]
    restart: unless-stopped
```

En `volumes:` agrega: `grafana-data:` y `loki-data:`.

> **Por qué así:** todo va en `microservices-network` para que Prometheus alcance a los servicios por nombre. La contraseña de Grafana sale de una variable de entorno (agrégala a `.env.example` y no subas el `.env`).

## 6. Paso 4 – Prometheus

**`observability/prometheus/prometheus.yml`** – siete targets, con el **API Gateway primero** y nombres de servicio (nunca IPs). Los puertos son los reales del proyecto:

```
global:
  scrape_interval: 15s
rule_files:
  - /etc/prometheus/rules.yml
scrape_configs:
  - job_name: 'api-gateway'
    metrics_path: /metrics
    static_configs: [{ targets: ['api-gateway:8080'] }]
  - job_name: 'empleados-service'
    metrics_path: /actuator/prometheus
    static_configs: [{ targets: ['empleados-service:8080'] }]
  - job_name: 'departamentos-service'
    metrics_path: /metrics
    static_configs: [{ targets: ['departamentos-service:8081'] }]
  - job_name: 'perfiles-service'
    metrics_path: /metrics
    static_configs: [{ targets: ['perfiles-service:8083'] }]
  - job_name: 'notificaciones-service'
    metrics_path: /metrics
    static_configs: [{ targets: ['notificaciones-service:8084'] }]
  - job_name: 'vacaciones-service'
    metrics_path: /metrics
    static_configs: [{ targets: ['vacaciones-service:8085'] }]
  - job_name: 'auth-service'
    metrics_path: /actuator/prometheus
    static_configs: [{ targets: ['auth-service:8086'] }]
```

Hasta que cada compañero instrumente su servicio, su target aparecerá en **DOWN**. Es normal.

**`rules.yml` – métricas unificadas.** Cada librería llama distinto a sus métricas (Micrometer, `prom-client`, `prometheus-net`…). Para que el dashboard y las alertas de P1 no dependan de eso, tú defines nombres

únicos con *recording rules*:

```
        groups:
          - name: unificadas
            rules:
              - record: service:http_requests:rate1m
                expr: >
                  sum by (job) (rate(http_requests_total[1m]))
                  or sum by (job) (rate(http_server_requests_seconds_count[1m]))
                  or sum by (job) (rate(http_requests_received_total[1m]))
              - record: service:http_errors5xx:ratio1m
                expr: >
                  ( sum by (job) (rate(http_requests_total{status=~"5.."}[1m]))
                    or sum by (job) (rate(http_server_requests_seconds_count{status=~"5.."}
[1m]))
                    or sum by (job) (rate(http_requests_received_total{code=~"5.."}[1m])) )
                  / service:http_requests:rate1m
              - record: service:http_latency_avg_seconds:rate1m
                expr: >
                  sum by (job) (rate(http_request_duration_seconds_sum[1m]))
                  / sum by (job) (rate(http_request_duration_seconds_count[1m]))
                  or sum by (job) (rate(http_server_requests_seconds_sum[1m]))
                  / sum by (job) (rate(http_server_requests_seconds_count[1m]))
```

> Son tres familias de nombres: `prom-client`/Go/Python (`http_requests_total`), Micrometer/Java (`http_server_requests_seconds_*`) y `prometheus-net`/.NET (`http_requests_received_total`). Cuando los demás terminen, revisa que cada una aparezca y ajusta las expresiones si alguna librería usó otros nombres.

## 7. Paso 5 – Loki y Promtail

**`observability/loki/loki-config.yml`** (almacenamiento local, un solo nodo):

```
auth_enabled: false
server:
  http_listen_port: 3100
common:
  path_prefix: /loki
  replication_factor: 1
  ring: { kvstore: { store: inmemory } }
  storage:
    filesystem: { chunks_directory: /loki/chunks, rules_directory: /loki/rules }
schema_config:
  configs:
    - from: 2024-01-01
      store: tsdb
      object_store: filesystem
      schema: v13
      index: { prefix: index_, period: 24h }
limits_config:
  allow_structured_metadata: true
```

**`observability/promtail/promtail-config.yml`** – descubre los contenedores por el socket de Docker y extrae los campos del JSON:

```
server:
  http_listen_port: 9080
positions:
  filename: /tmp/positions.yaml
clients:
  - url: http://loki:3100/loki/api/v1/push
scrape_configs:
  - job_name: docker-containers
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
        refresh_interval: 5s
    relabel_configs:
      - source_labels: ['__meta_docker_container_name']
        regex: '/(.*)'
        target_label: container
      - source_labels: ['__meta_docker_container_label_com_docker_compose_service']
        target_label: service
    pipeline_stages:
      - json:
          expressions: { level: level, traceId: traceId }
      - labels: { level: }
```

> El `pipeline_stages` solo es útil cuando los servicios ya emitan JSON; con logs en texto plano no falla, simplemente no extrae nada. Etiquetamos `level` y dejamos `traceId` como campo del mensaje (no como etiqueta) para no crear demasiadas series.

## 8. Paso 6 – Provisioning de Grafana

**`datasources/datasources.yml`** – tres fuentes ya configuradas al arrancar:

```yaml
apiVersion: 1
datasources:
  - name: Prometheus
    uid: prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
  - name: Loki
    uid: loki
    type: loki
    access: proxy
    url: http://loki:3100
  - name: Zipkin
    uid: zipkin
    type: zipkin
    access: proxy
    url: http://zipkin:9411
```

**`dashboards/dashboards.yml`** – le dice a Grafana dónde buscar los JSON (P1 pondrá el suyo ahí):

```yaml
apiVersion: 1
providers:
  - name: 'default'
    folder: 'Reto 8'
    type: file
    options:
      path: /etc/grafana/provisioning/dashboards
```

## 9. Paso 7 – Instrumentar el API Gateway (Node + Express)

El gateway no tiene base de datos ni broker, así que su `/health` es sencillo. Hoy depende de `express`, `http-proxy-middleware` y `jsonwebtoken`.

**1) Dependencias** (en `api-gateway/`):

```
npm install prom-client winston @opentelemetry/api @opentelemetry/sdk-node \
  @opentelemetry/auto-instrumentations-node @opentelemetry/exporter-zipkin
```

**2) `src/tracing.js`** – **se carga antes que Express**, para que la instrumentación automática enganche `http`:

```
        // Arranca OpenTelemetry. Debe cargarse ANTES de la app (node --require).
        const { NodeSDK } = require('@opentelemetry/sdk-node');
        const { ZipkinExporter } = require('@opentelemetry/exporter-zipkin');
        const { getNodeAutoInstrumentations } = require('@opentelemetry/auto-instrumentations-
node');

        const sdk = new NodeSDK({
          serviceName: process.env.OTEL_SERVICE_NAME || 'api-gateway',
          traceExporter: new ZipkinExporter({
            url: process.env.OTEL_EXPORTER_ZIPKIN_ENDPOINT ||
'http://zipkin:9411/api/v2/spans',
          }),
          // Propagación W3C (tracecontext + baggage) viene por defecto.
          instrumentations: [getNodeAutoInstrumentations()],
        });
        sdk.start();
        process.on('SIGTERM', () => sdk.shutdown());
```

**3)** `src/observabilidad.js` – métricas con **los nombres del contrato** (`http_requests_total`, `http_request_duration_seconds`), logger JSON con `traceId` y `/health`:

```
        const client = require('prom-client');
        const winston = require('winston');
        const { trace } = require('@opentelemetry/api');

        client.collectDefaultMetrics();                    // CPU y memoria del proceso
        const peticiones = new client.Counter({
          name: 'http_requests_total', help: 'Peticiones HTTP', labelNames: ['method',
'status'] });
        const duracion = new client.Histogram({
          name: 'http_request_duration_seconds', help: 'Latencia HTTP', labelNames: ['method',
'status'] });

        // Agrega el traceId activo a cada línea de log.
        const conTrace = winston.format((info) => {
          const span = trace.getActiveSpan();
          if (span) info.traceId = span.spanContext().traceId;
          return info;
        });
        const logger = winston.createLogger({
          level: 'info',
          format: winston.format.combine(
            conTrace(),
            winston.format.timestamp(),
            winston.format((i) => { i.service = 'api-gateway'; return i; })(),
            winston.format.json()),
          transports: [new winston.transports.Console()],
        });

        function middlewareMetricas(req, res, next) {
          const fin = duracion.startTimer();
          res.on('finish', () => {
            const l = { method: req.method, status: String(res.statusCode) };
            peticiones.inc(l); fin(l);
          });
          next();
        }

        function registrarRutas(app) {
          app.get('/metrics', async (_req, res) => {
            res.set('Content-Type', client.register.contentType);
            res.end(await client.register.metrics());
          });
          app.get('/health', (_req, res) =>
            res.status(200).json({ status: 'UP', service: 'api-gateway', checks: {} }));
        }
        module.exports = { logger, middlewareMetricas, registrarRutas };
```

**4) En `src/server.js`:** importa el módulo, registra `app.use(middlewareMetricas)` **antes** de las rutas del proxy, llama a `registrarRutas(app)` y reemplaza los `console.log` por `logger.info(...)`. Revisa que

`/health` y `/metrics` **no pasen por la validación JWT** (mira `security.js`); si el gateway exige token en todo, añade esas dos rutas a la lista de rutas públicas.

**5)** **`package.json`** **:** cambia el script a `"start": "node --require ./src/tracing.js src/server.js"`. Revisa también que el **`CMD`** **del Dockerfile** use `npm start` o el mismo comando; y que el Dockerfile ya no diga `EXPOSE 3000` (el gateway escucha en 8080).

**6) Variables en el compose** (bajo `api-gateway:`):

```
      OTEL_SERVICE_NAME: api-gateway
      OTEL_EXPORTER_ZIPKIN_ENDPOINT: http://zipkin:9411/api/v2/spans
      OTEL_PROPAGATORS: tracecontext,baggage
```

## 10. Paso 8 – Contrato común (`docs/contrato-observabilidad.md`)

Es el documento que desbloquea a P1, P2 y P3. Debe fijar, como mínimo:

| Tema | Regla |
|---|---|
| Red | `microservices-network` |
| Variables OTel | `OTEL_SERVICE_NAME=<servicio>`, `OTEL_EXPORTER_ZIPKIN_ENDPOINT=http://zipkin:9411/api/v2/spans`, `OTEL_PROPAGATORS=tracecontext,baggage` |
| Ruta de métricas | Java: `/actuator/prometheus`. Node, Go, Python y .NET: `/metrics` |
| Métricas | Node, Go y Python: `http_requests_total{method,status}` y `http_request_duration_seconds`. Java y .NET: las que emite su librería (ya están contempladas en `rules.yml`) |
| `/health` | `GET /health` → `{status, service, checks:{database, messageBroker}}`. 200 si UP y **503** si DOWN. En Java: `/actuator/health` ajustado a ese formato |
| Log JSON | Campos `timestamp`, `level`, `service`, `traceId`, `message` |
| Broker | El que publica inyecta la cabecera **`traceparent`** en el mensaje. El que consume la extrae para continuar la traza |
| Puertos reales | gateway 8080, empleados 8080, departamentos 8081, perfiles 8083, notificaciones 8084, vacaciones 8085, auth 8086 |

Incluye también el nombre de las URLs: `http://zipkin:9411`, `http://prometheus:9090`, `http://loki:3100`.

## 11. Paso 9 – Validación (hazlo antes de entregar)

1. `docker compose config --quiet` – valida que el compose es correcto.
2. `docker compose up -d --build` y luego `docker compose ps`: todo en *Up* (los servicios aún sin instrumentar siguen sirviendo, solo no exponen métricas).
3. **Prometheus:** `http://localhost:9091/targets`. El gateway debe estar **UP**; los demás pueden estar en DOWN.
4. **Reglas:** en `http://localhost:9091/rules` deben cargar sin error las tres series `service:*`.
5. **Grafana:** `http://localhost:3000` (admin / contraseña de `.env`). En *Connections → Data sources* deben aparecer Prometheus, Loki y Zipkin, ya configurados.

6. **Zipkin:** `http://localhost:9411`. Haz un par de peticiones al gateway (`curl http://localhost:8080/health`) y busca trazas con `serviceName=api-gateway`.
7. **Loki:** en Grafana → Explore → Loki: `{service="api-gateway"}`. Deben aparecer los logs en JSON.
8. **Persistencia:** `docker compose restart grafana` y comprueba que no se pierde nada.
9. **Con 8 GB, si todo se pone lento:** apaga bases de datos y servicios que no uses, o prueba solo la observabilidad más el gateway.

### Compatibilidad con la CI del Reto 7 (muy importante)

El pipeline E2E hace `docker compose -f docker-compose.yml -f docker-compose.e2e.yml up -d` sobre **todo** el compose. Al agregar cinco servicios, la CI también los levantaría (más RAM y puertos ocupados: el 9091, 3000 y 9411).

- Antes de entregar, lanza **un build** de la CI y comprueba que sigue verde.
- Si choca por puertos o memoria, en `docker-compose.e2e.yml` quita los puertos publicados de los servicios de observabilidad (`ports: !reset []`) o súmalos a un `profiles` para que la CI no los inicie. Pruébalo antes de dejarlo, porque la sintaxis depende de la versión de Compose.
- **Avisa al equipo** antes de hacer push: el cambio toca un archivo compartido.

## 12. Paso 10 – README y entrega

1. Tu sección del README: investigación (Paso 1), diagrama, justificación Zipkin, tabla de puertos y URLs, y cómo levantar el stack.
2. Commit y push a `main`, en commits separados y claros, por ejemplo:
   - `feat(observabilidad): stack prometheus, grafana, loki, promtail y zipkin`
   - `feat(gateway): metricas, trazas y logs json`
   - `docs(observabilidad): contrato y diagrama`
3. Avisa a **P2** (siguiente turno) de que ya pueden hacer `git pull`.

## 13. Cuándo está terminado

- [ ] El compose levanta todo sin errores y con una sola orden.
- [ ] Prometheus ve el gateway en UP, con los 7 jobs configurados.
- [ ] Grafana arranca con Prometheus, Loki y Zipkin ya cargados.
- [ ] El gateway aparece en Zipkin con su nombre, y sus logs salen en Loki.
- [ ] `docs/contrato-observabilidad.md` publicado.
- [ ] Un build de la CI del Reto 7 sigue en verde.
- [ ] README con investigación, diagrama y justificación.

## 14. Problemas frecuentes

| Síntoma | Causa probable |
|---|---|
| Prometheus no arranca | Error de sintaxis en YAML: valida con `docker compose logs prometheus`. |
| Target del gateway en DOWN | El gateway pide JWT en `/metrics`, o el puerto no es 8080. |
| Promtail sin logs | Falta el montaje de `docker.sock` o los contenedores no son del mismo proyecto. |

| Síntoma | Causa probable |
| --- | --- |
| Grafana no ve Loki | Loki aún arrancando; espera 30 s y recarga. |
| Zipkin sin trazas del gateway | `tracing.js` no se carga antes de Express (revisa `--require`). |
| Puerto 9090 ocupado | Es Jenkins; por eso Prometheus usa `9091` en el host. |